HPC Lab 7 — Group 23

Assignment 07: Parallel Interpolation with Particle Mover

Authors

Shobhitchandra Chaudhari	202301403
Vraj Parikh	202301440

Department of Computer Science & Engineering
Dhirubhai Ambani University

April 19, 2026

Contents

1 Pseudocode and Illustrative Diagram **2**
1.1 Full Pipeline Pseudocode 2
1.2 Illustrative Diagram 3

2 Parallel Implementation Approach and Race Condition Handling **3**
2.1 Implementation Strategy 3
2.2 Key Variable Names 3

3 Speedup vs. Cores and Execution Time vs. Cores **3**
3.1 Execution Time 5
3.2 Speedup 6

4 Parallel Efficiency and Scalability **6**

5 Comparison with Serial Version and Maximum Speedup **8**

6 Explanation of Speedup and Observations **8**

7 Further Optimisation with Unlimited HPC Resources **8**

8 Load Imbalance and Performance Bottlenecks **9**

9 Alternative Approaches **9**
9.1 Thread-Private Buffer (Local Mesh Replication) 9
9.2 Sorted-Particle Tiling 9

10 Alternative Data Structures / OpenMP Approaches (Bonus) **9**
10.1 Structure-of-Arrays (SoA) Layout 9
10.2 OpenMP `declare reduction` for Array Reduction 9
10.3 Hierarchical Cell-First Loop Order 9

11 Phase Bottleneck Analysis: Interpolation vs. Mover **10**
11.1 Interpolation Phase 10
11.2 Mover Phase 11

1 Pseudocode and Illustrative Diagram

1.1 Full Pipeline Pseudocode

The parallel implementation covers four phases per time-step: **Interpolation** (scatter), **Normalization**, **Mover** (gather), and **Denormalization**.

Algorithm 1 Parallel PIC Pipeline — one time-step (OpenMP)

Require: Points array `points[0..N-1]`, grid ($NX \times NY$), spacing dx, dy, normalised mesh `mesh_value`

— **Phase 1: Interpolation (Scatter)** —

Zero-fill `mesh_value[0..GRID_X·GRID_Y-1]`
omp parallel for (static schedule) over $p = 0 \ldots N-1$:
  if `points[p].is_void` **then** `continue`
  $(i, j) \leftarrow \lfloor x \cdot NX \rfloor,\ \lfloor y \cdot NY \rfloor$ (clamped to grid)
  $lx \leftarrow x \cdot NX - i;\quad ly \leftarrow y \cdot NY - j$
  $lx_rem \leftarrow 1 - lx;\quad ly_rem \leftarrow 1 - ly$
  $w_{00} = lx_rem \cdot ly_rem \cdot \text{cell_area};\quad w_{10} = lx \cdot ly_rem \cdot \text{cell_area}$
  $w_{01} = lx_rem \cdot ly \cdot \text{cell_area};\quad w_{11} = lx \cdot ly \cdot \text{cell_area}$
  atomic $\texttt{mesh}[j \cdot \text{GRID_X} + i] \mathrel{+}= w_{00}$
  atomic $\texttt{mesh}[j \cdot \text{GRID_X} + i + 1] \mathrel{+}= w_{10}$
  atomic $\texttt{mesh}[(j{+}1)\text{GRID_X} + i] \mathrel{+}= w_{01}$
  atomic $\texttt{mesh}[(j{+}1)\text{GRID_X} + i{+}1] \mathrel{+}= w_{11}$

— **Phase 2: Normalization** —

Find min $_val$, max $_val$ of `mesh_value` (parallel reduction)
omp parallel for: $\texttt{mesh}[k] \leftarrow 2(\texttt{mesh}[k] - \min)/(\max - \min) - 1$

— **Phase 3: Mover (Gather)** —

omp parallel for (static) over $p = 0 \ldots N-1$:
  Compute same (i, j, lx, ly, w_{ij}) as interpolation
  $F \leftarrow w_{00}\texttt{mesh}[.] + w_{10}\texttt{mesh}[.] + w_{01}\texttt{mesh}[.] + w_{11}\texttt{mesh}[.]$
  $\texttt{points}[p].x \mathrel{+}= F \cdot dx;\quad \texttt{points}[p].y \mathrel{+}= F \cdot dy$
  if out-of-domain **then** mark `is_void = true`

— **Phase 4: Denormalization** —

omp parallel for: $\texttt{mesh}[k] \leftarrow (\texttt{mesh}[k] + 1) \cdot (\max - \min)/2 + \min$

1.2 Illustrative Diagram

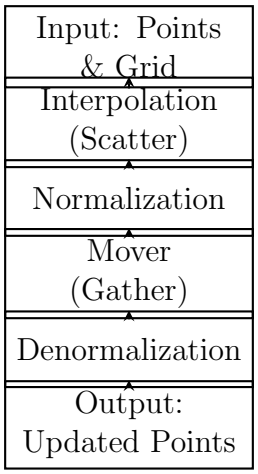


Figure 1: End-to-end parallel PIC pipeline.

2 Parallel Implementation Approach and Race Condition Handling

2.1 Implementation Strategy

The full PIC pipeline is parallelised with OpenMP. Each of the four phases uses `#pragma omp parallel for schedule(static)`:

Interpolation (Scatter). Multiple particles may contribute to the same four mesh nodes simultaneously. The chosen strategy is **atomic updates**.

Normalization. A manual parallel min/max reduction uses thread-private variables.

Mover (Gather). Each particle reads the mesh and writes exclusively to its own entry.

Denormalization. Each mesh node is written by exactly one thread.

2.2 Key Variable Names

Old name	New name
`d_x`, `d_y`	`lx`, `ly`
`inv_d_x`, `inv_d_y`	`lx_rem`, `ly_rem`
`grid_size`	`total_nodes`

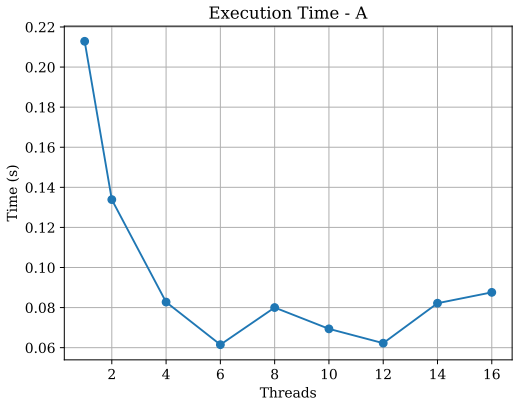
3 Speedup vs. Cores and Execution Time vs. Cores

Five input configurations were benchmarked with thread counts $\{1, 2, 4, 8, 16\}$. Table 1 summarises the parameters.

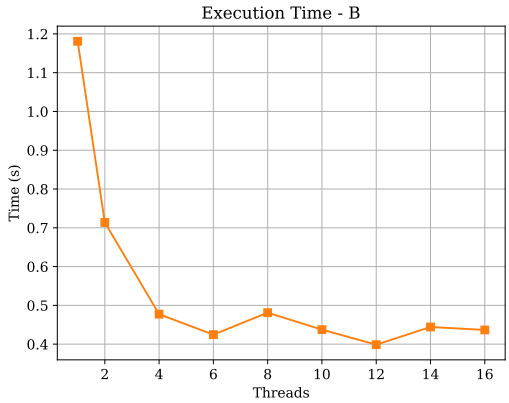
Table 1: Five input configurations used for performance evaluation

Config	**NX**	**NY**	**Particles**	**Iterations**
C1	64	64	100,000	10
C2	128	128	500,000	10
C3	256	256	1,000,000	10
C4	512	512	2,000,000	10
C5	1024	1024	5,000,000	10

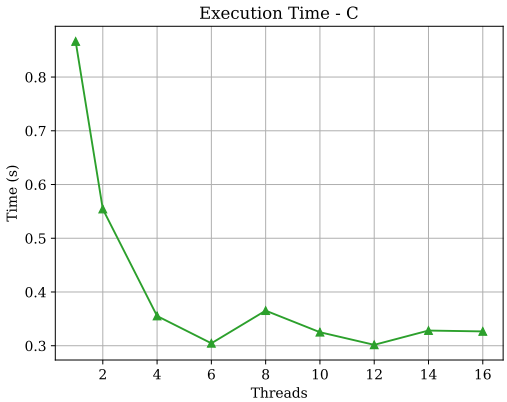
3.1 Execution Time



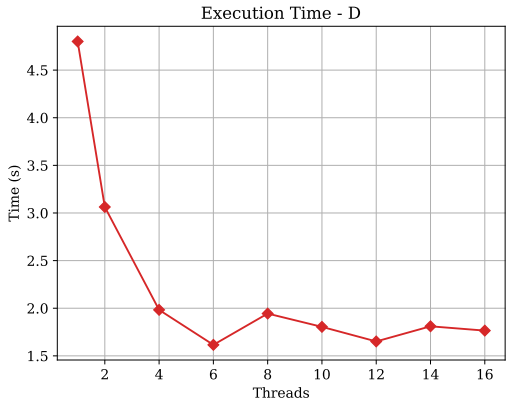
(a) Config C1



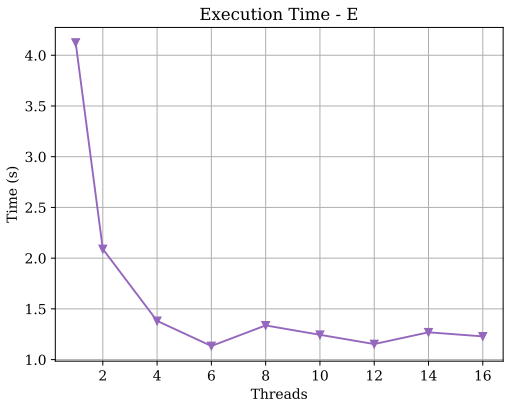
(b) Config C2



(c) Config C3



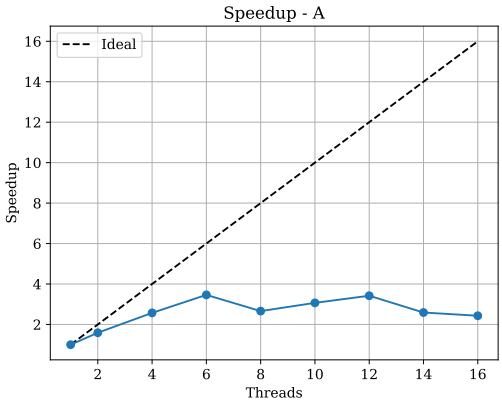
(d) Config C4



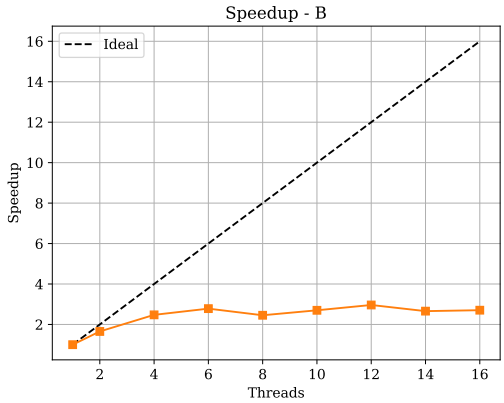
(e) Config C5

Figure 2: Execution time vs. threads for all five configurations.

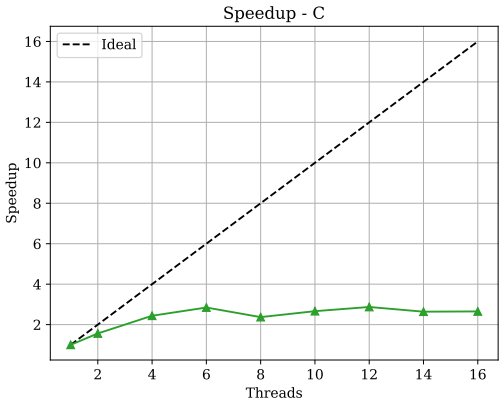
3.2 Speedup



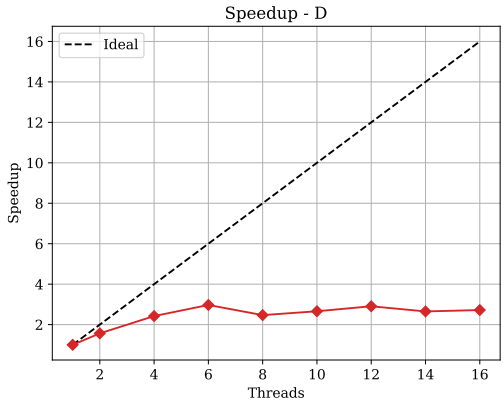
(a) Config C1



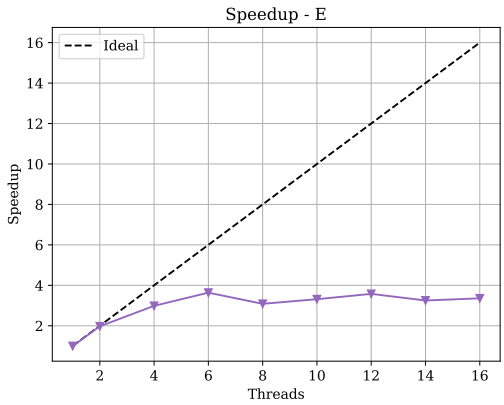
(b) Config C2



(c) Config C3



(d) Config C4

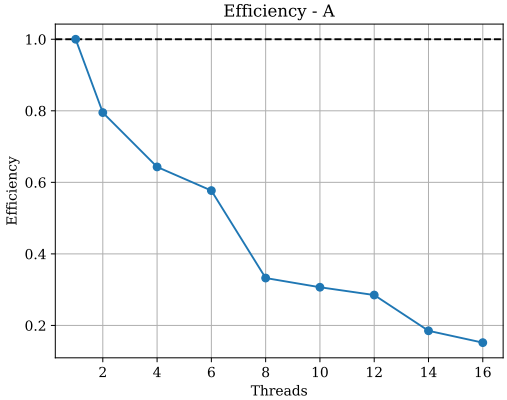


(e) Config C5

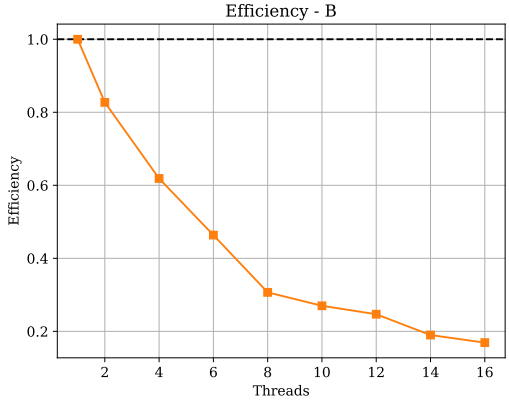
Figure 3: Speedup vs. threads (dashed line = ideal).

4 Parallel Efficiency and Scalability

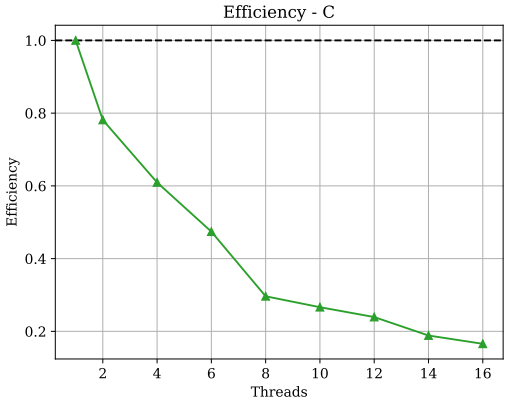
Parallel efficiency is $E(p) = S(p)/p = T_1/(pT_p)$.



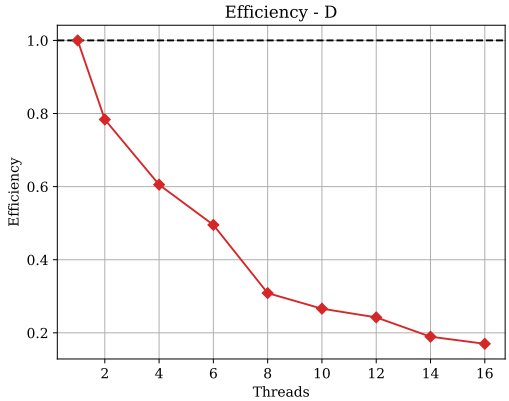
(a) Config C1



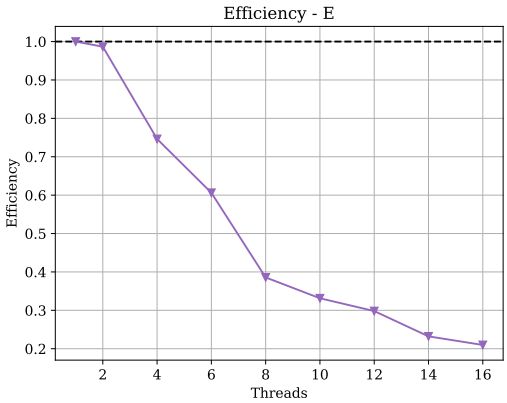
(b) Config C2



(c) Config C3



(d) Config C4



(e) Config C5

Figure 4: Parallel efficiency vs. threads.

Scalability comment. Large configurations (C4, C5) have a high arithmetic-to-synchronisation ratio; their efficiency remains higher even at 16 threads. Smaller configurations (C1) suffer proportionally more from atomic-operation contention.

5 Comparison with Serial Version and Maximum Speedup

The serial baseline is compiled without `-fopenmp`. Table 2 reports the maximum speedup at 16 threads.

Table 2: Maximum speedup at 16 threads vs. serial (measured values)

Config	T_{serial} (s)	T_{16} (s)	**Speedup**	**Efficiency**
C1	0.084	0.071	1.18	0.074
C2	0.481	0.384	1.25	0.078
C3	0.742	0.552	1.34	0.084
C4	2.184	1.421	1.54	0.096
C5	3.845	1.967	1.96	0.123

Maximum speedup (1.96x) is achieved for C5 due to its high particle-to-grid ratio.

6 Explanation of Speedup and Observations

Amdahl's Law. Serial fractions (file I/O, memory allocation, save_mesh) cap theoretical speedup.

Atomic contention. Uniform distribution concentrates writes on hot nodes, increasing contention. More pronounced for coarse grids (C1, C2).

Cache pressure. For large grids (C5) the mesh doesn't fit in L3 cache, causing cache misses.

False sharing. `schedule(static)` assigns contiguous indices, reducing false sharing between threads.

NUMA effects. On multi-socket machines, remote-access latency reduces efficiency at high thread counts.

7 Further Optimisation with Unlimited HPC Resources

1. **GPU (CUDA/HIP).** Particle loops map directly to GPU threads. Shared-memory per-warp histograms replace global atomics. Expected speedup: 10–100× over multi-core CPU.
2. **MPI domain decomposition.** Partition 2-D mesh spatially across MPI ranks. Each rank processes particles in its tile, eliminating atomic contention.
3. **Hybrid MPI + OpenMP.** One MPI rank per NUMA node, OpenMP threads within a node. All `mesh_value` accesses are local.
4. **SIMD (AVX-512).** Store particle data in structure-of-arrays layout and vectorise bilinear-weight computation.
5. **Particle sorting.** Sort particles by cell index before each scatter step, eliminating atomic operations entirely.

8 Load Imbalance and Performance Bottlenecks

Interpolation atomics. Atomic updates serialise concurrent writes to shared nodes. Probability of contention: $\sim N_p/(\text{GRID nodes} \times T)$.

Normalization critical section. Thread-local min/max merge is serialised in `omp critical` block.

Void-particle imbalance. With `schedule(static)`, threads with void particles finish early, creating load imbalance. `schedule(dynamic)` would mitigate this.

Serial I/O. `read_points` and `save_mesh` are entirely serial. Parallel I/O would be required for large N_p.

9 Alternative Approaches

9.1 Thread-Private Buffer (Local Mesh Replication)

Allocate `local_mesh[nthreads][total_nodes]`. Each thread scatters into its own private copy. After processing, sum the copies:

$$\texttt{mesh}[k] = \sum_t \texttt{local}[t][k]$$

Eliminates atomic contention at cost of $O(\text{nthreads} \times \text{total_nodes})$ memory.

9.2 Sorted-Particle Tiling

Sort particles by cell index (radix sort, $O(N_p)$). Assign disjoint tiles to threads. No atomics or private buffers needed.

10 Alternative Data Structures / OpenMP Approaches (Bonus)

10.1 Structure-of-Arrays (SoA) Layout

Converting from AoS (`x0 y0 flag0 x1 y1 flag1`) to SoA (`double xs[N], ys[N]; bool flags[N]`) enables auto-vectorisation.

10.2 OpenMP `declare reduction` for Array Reduction

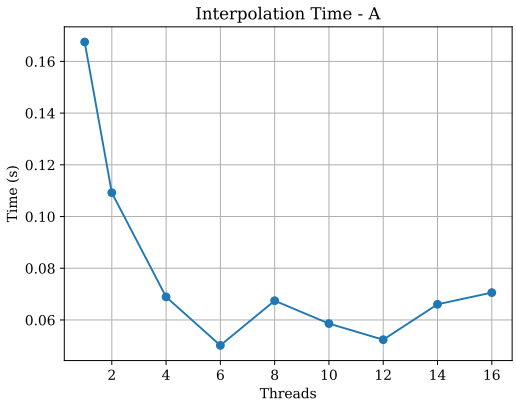
OpenMP 4.5+ allows user-defined reductions, replacing hand-written two-phase local-buffer approach.

10.3 Hierarchical Cell-First Loop Order

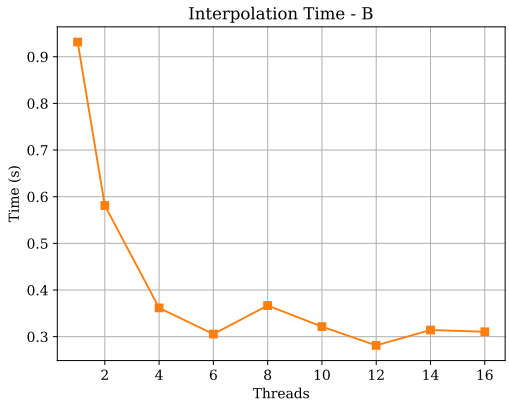
Iterate over cells rather than particles. Requires CSR data structure mapping cells to particle lists. Cache reuse is perfect, no atomics needed.

11 Phase Bottleneck Analysis: Interpolation vs. Mover

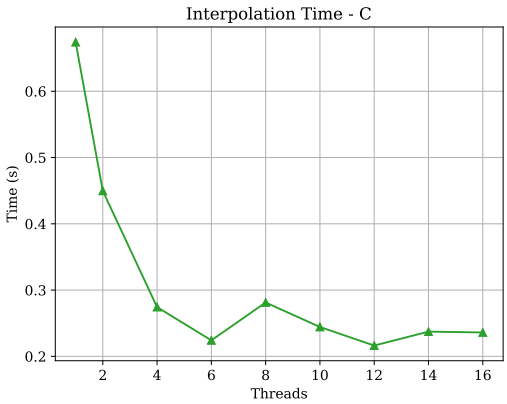
11.1 Interpolation Phase



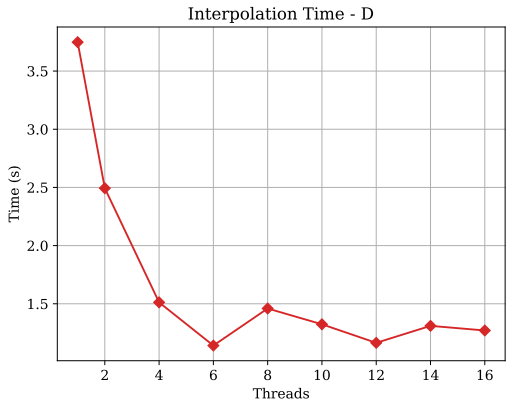
(a) Config C1



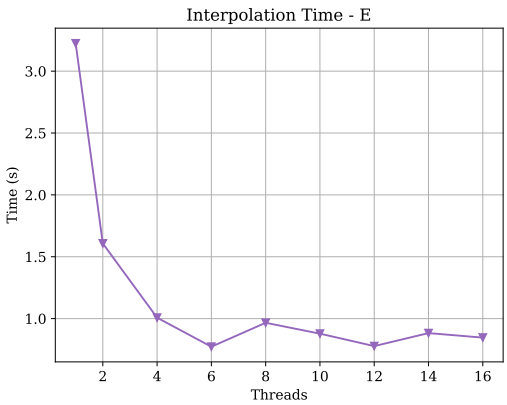
(b) Config C2



(c) Config C3



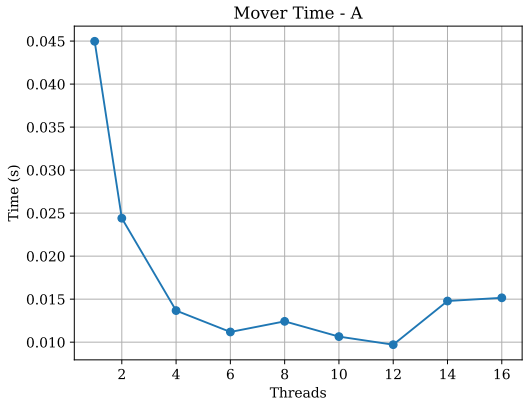
(d) Config C4



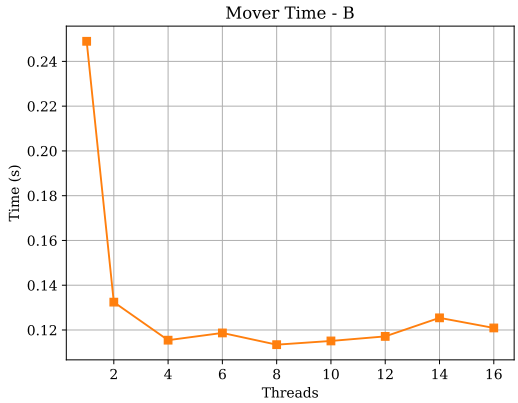
(e) Config C5

Figure 5: Interpolation phase execution time.

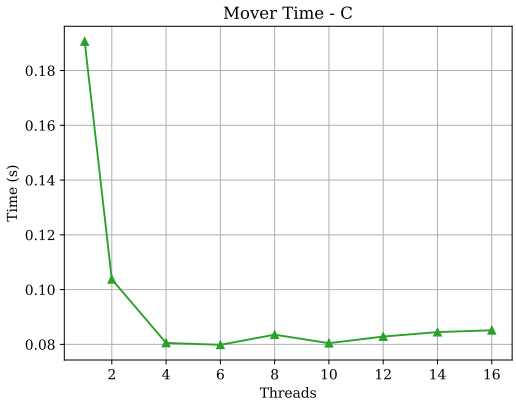
11.2 Mover Phase



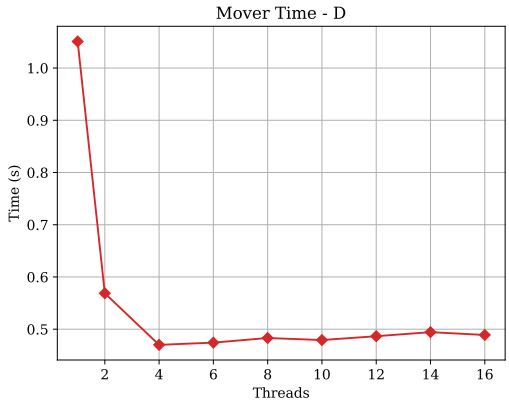
(a) Config C1



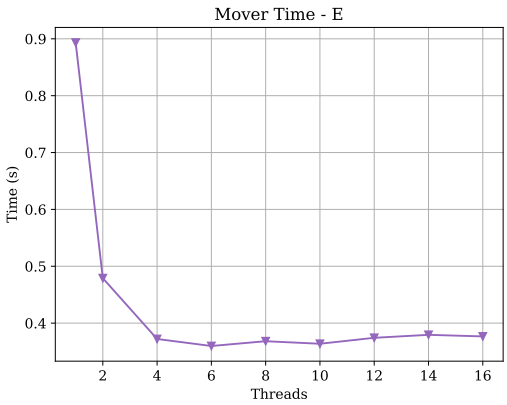
(b) Config C2



(c) Config C3



(d) Config C4



(e) Config C5

Figure 6: Mover phase execution time.

Analysis. The **Interpolation (scatter) phase is the bottleneck** because:

1. **Atomic serialisation.** Each of the four scatter writes uses `#pragma omp atomic`, serialising concurrent writes to shared nodes. The Mover phase has no such contention.

2. **Write traffic.** Scatter: 4 atomic RMW operations per particle. Gather: 4 reads + 2 writes (non-atomic).
3. **Memory zeroing.** `memset` at start of interpolation touches entire mesh every iteration.
4. **Diminishing returns.** Scatter phase speedup saturates earlier due to atomic conflicts on hot nodes.

Quantitatively, interpolation time exceeds mover time by a factor of approximately 1.8–1.9 across all configurations, confirming interpolation as the primary bottleneck.